Faculty of Computing

CS 416: Large Language Models (LLMs)

Class: BESE 13 (Gp1 & Gp2)

Lab 08: Open Ended Lab

Date: 17 Mar 2026

Time: 10:00 AM–1:00 PM & 2:00 PM – 5:00 PM

Instructor: Dr. Faisal Shafait & Dr. Momina Mohtesum

Lab Engineer: Iram Tariq Bhatti

Lab 08: Open Ended Lab

Learning Outcome

CLO-3: Apply transfer learning and prompt engineering to solve various problems using large language models (C-3)

Tools / Software Requirements

Python (>= 3.7), IDE: PyCharm (preferred) or VSCode or Google Colab

- TensorFlow or PyTorch (student's choice)
- NumPy
- Dataset: reviews.txt and labels.txt (provided)
- Any additional libraries of student's choice — must be clearly stated in submission

Contents

1. Objectives
2. Core Ideas
3. Tasks
4. Deliverable
5. Additional Resources

1) Objectives

By the end of this lab, students will be able to:

- Implement a complete text preprocessing and encoding pipeline for sentiment analysis
- Apply an LSTM-based recurrent neural network architecture to solve a binary classification task
- Execute model training with batch processing and produce checkpoint-based saving
- Demonstrate model performance through inference on a held-out test set
- Independently design and justify an end-to-end deep learning solution using a library of their choice

2) Core Ideas

Sentiment Analysis

Sentiment analysis is a text classification task that assigns polarity labels (positive/negative) to textual inputs such as movie reviews. It is a foundational NLP problem that benchmarks a model's ability to capture semantic meaning from natural language.

Word Embeddings

Rather than one-hot encoding words (which produces extremely sparse, high-dimensional vectors), an embedding layer maps each integer-encoded word to a dense lower-dimensional vector. The network learns these embeddings during training, allowing it to encode semantic relationships between words.

LSTM (Long Short-Term Memory)

LSTMs are a type of recurrent neural network designed to capture long-range dependencies in sequential data. Unlike simple RNNs, LSTMs use gating mechanisms (input, forget, output gates) to selectively retain or discard information across time steps, making them well-suited for variable-length text sequences.

Padding and Feature Engineering

Neural networks require fixed-length inputs. Reviews of varying lengths are standardized to 200 tokens: reviews shorter than 200 are left-padded with zeros; reviews longer than 200 are truncated to the first 200 tokens.

Train / Validation / Test Split

Data is partitioned into training (80%), validation (10%), and test (10%) sets. The training set is used for gradient updates, the validation set for monitoring overfitting during training, and the test set for final unbiased evaluation of model performance.

3) Tasks

Since this is an Open Ended Lab, students have freedom in their choice of framework (TensorFlow or PyTorch), architecture variants, and hyperparameter settings. All assumptions must be clearly documented in the notebook.

Task 1: Implement Data Loading and Execute Text Preprocessing for Sentiment Analysis

Load and preprocess the dataset:

- Read reviews.txt and labels.txt
- Remove all punctuation from the review text
- Split reviews by newline delimiter and tokenize into individual words
- Verify the data by printing sample reviews and their lengths

Task 2: Implement Vocabulary Encoding to Produce Integer-Mapped Reviews

Build a vocabulary-to-integer mapping:

- Create a dictionary that maps each unique word to a unique integer starting at 1 (not 0, since 0 is reserved for padding)
- Convert all reviews from word sequences to integer sequences (reviews_ints)
- Use Counter to analyze the distribution of review lengths
- Remove any zero-length reviews from the dataset

Task 3: Apply Label Encoding to Produce Binary Sentiment Targets

Convert string labels to binary integers:

- Map 'positive' to 1 and 'negative' to 0
- Verify label distribution across the dataset
- Ensure labels are aligned with their corresponding reviews after filtering

Task 4: Implement Padded Feature Vector Construction for Fixed-Length Input

Create a features array for network input:

- Set sequence length to 200 tokens per review
- Left-pad shorter reviews with zeros
- Truncate longer reviews to the first 200 tokens
- Verify feature matrix shape and spot-check values against expected output

Task 5: Execute Dataset Splitting to Produce Train, Validation, and Test Sets

Partition the dataset using split_frac = 0.8:

- Training set: first 80% of features and labels
- Validation set: first half of the remaining 20%
- Test set: second half of the remaining 20%
- Verify final shapes: Train (20000, 200), Validation (2500, 200), Test (2501, 200)

Task 6: Implement and Apply an LSTM-Based Sentiment Classification Model

Build the neural network model using TensorFlow or PyTorch (student's choice):

- Embedding layer: maps integer tokens to dense vectors
- LSTM layer(s): capture sequential dependencies across the review
- Dropout regularization: applied to LSTM output
- Sigmoid output layer: single unit for binary classification

Clearly state all hyperparameters (embedding size, hidden units, number of layers, dropout rate) and justify your choices.

Task 7: Execute Model Training with Batch Processing and Checkpoint Saving

Train the model using the get_batches() function provided:

- Use batch size of 100
- Run forward pass, compute loss (MSE or BCE), and execute backpropagation
- Monitor training and validation loss across epochs
- Save model checkpoints to the ./checkpoints/ directory

Task 8: Demonstrate Model Performance by Executing Inference on the Test Set

Evaluate the trained model:

- Run inference on the test set using saved checkpoints
- Compute and report test accuracy
- Demonstrate predictions on at least 3 sample reviews
- Briefly discuss whether the model generalizes well and identify any patterns in misclassification

For the details of each task, please refer to the attached notebook.

4) Deliverable

Submit the completed .ipynb notebook with all tasks implemented. Since this is an open ended lab, clearly mention all assumptions made while designing your solution. Do not submit zipped folders. Submit a single .ipynb file on LMS before the deadline.

5) Additional Resources

- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- TensorFlow LSTM documentation: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
- PyTorch LSTM documentation: https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html
- Maas, A., et al. (2011). Learning Word Vectors for Sentiment Analysis. ACL.